pandas 2.0 and the Arrow revolution (part I)

datapythonista.me/blog/pandas-20-and-the-arrow-revolution-part-i

Marc Garcia February 17, 2023

Introduction

At the time of writing this post, we are in the process of releasing pandas 2.0. The project has a large number of users, and it's used in production quite widely by personal and corporate users. This large use based forces us to be conservative and make us avoid most big changes that would break existing pandas code, or would change what users already know about pandas. So, most changes to pandas, while they are important, they are quite subtle. Most of our changes are bug fixes, code improvements and clean up, performance improvements, keep up to date with our dependencies, small changes that make the API more consistent, etc.

A recent change that may seem subtle and it's easy to not be noticed, but it's actually very important is the new Apache Arrow backend for pandas data. To understand this change, let's quickly summarize how pandas works. The general idea is that before being able to do anything in pandas it is needed to load into memory the data of interest (using methods like `read_csv`, `read_sql`, `read_parquet`, etc). When loading data into memory it's required to decide how this data will be stored in memory. For simple data like integers of floats this is in general not so complicated, as how to represent a single item is mostly standard, and we just need arrays of the number of elements in our data. But for other types (such as strings, dates and times, categories, etc.) some decisions need to be made. Python is able to represent mostly anything, but Python data structures (lists, dictionaries, tuples, etc) are very slow and can't be used. So the data representation is not Python and is not standard, and an implementation needs to happen via Python extensions, usually implemented in C (also in C++, Rust and others). For many years, the main extension to represent arrays and perform operations on them in a fast way has been NumPy. And this is what pandas was initially built on.

While NumPy has been good enough to make pandas the popular library it is, it was never built as a backend for dataframe libraries, and it has some important limitations. A couple of examples are the poor support for strings and the lack of missing values. To get a more detailed idea of this topic, you can read Wes McKinney's article Apache Arrow and the 10 Things I Hate About pandas.

For some years now, while still relying heavely on NumPy, pandas has been slowly decoupling from it. A couple of important milestones were the addition of an API to implement Extension Arrays for pandas in 2018. Extension arrays allow pandas to work with custom data types. Not long after they were added to pandas, all existing complex types in pandas (e.g. categorical or datetime with timezone types) were moved to this interface. Another important milestone was the implementation of a string data type based on Arrow that started in 2020.

After these additions, in pandas 1.5 and 2.0 we are adding Apache Arrow support for all data types. Which is the topic of the rest of this article.

The basics

By default pandas will keep using the original types. As mentioned earlier, one of our top priorities is not breaking existing code or APIs. See these pandas 2.0 examples:

```
>>> import pandas
>>> pandas.__version__
'2.0.0.dev0+1578.g32f1faddc3'

>>> pandas.Series([1, 2, 3, 4])
0    1
1    2
2    3
3    4
dtype: int64

>>> pandas.Series(['foo', 'bar', 'foobar'])
0       foo
1       bar
2    foobar
dtype: object
1234567891011121314151 6
```

Nothing really changed. But we can change the `dtype` to use Arrow:

```
>>> pandas.Series([1, 2, 3, 4], dtype='int64[pyarrow]')
0    1
1    2
2    3
3    4
dtype: int64[pyarrow]

>>> pandas.Series(['foo', 'bar', 'foobar'], dtype='string[pyarrow]')
0       foo
1       bar
2    foobar
dtype: string
123456789101112
```

In the pandas 2.0 release candidates there was a `dtype_backend` option to let pandas know we want Arrow backed types by default. The option was confusing since not all operations support generating Arrow backed data yet, and it was removed. For I/O operators that support creating Arrow-backed data, there is a `dtype_backend` parameter:

```
import pandas

pandas.read_csv(fname, engine='pyarrow', dtype_backend='pyarrow')
123
```

Note that the engine is somehow independent of the backend. We can use PyArrow function (engine) to read CSV files while using columns with a NumPy data type (backend), and the other way round.

Why Arrow?

There are several advantages that Arrow provides, even for simple types. We are going to list some of them in the next sections.

Missing values

Representing missing values is a complex topic. In Python it's not, since everything is wrapped as a Python object, it's possible to mix different types in lists, and you can simply use the value `None` for any missing data. But when performance is important, data types are represented in the CPU representation, and can't be mixed with other types. For example, an unsigned integer of 8 bits, will represent 0 as 00000000 and 255 as 11111111, and our whole list of values needs to use that. Any bit representation corresponds to a number in the range 0 to 255, and it's not possible to represent a missing value unless using a sentinel value. For example, using 255 as the missing value in your code, and only allowing values from 0 to 254 for your type. But this is complex and tricky, and surely not ideal.

For the case of floating point numbers, the internal (CPU) representation is more complex, and there are actually some sentinel values already defined in the IEEE 754 standard, which CPUs implement, and are able to deal with efficiently. For this reason, historically, the approach of pandas to missing values has been to convert numbers to floating point if they were not already, and use `NaN` as the missing value. Converting integer values to floating point notation to support missing value is again not ideal, and has side effects.

More recently, after extension arrays were added to pandas, it was possible for pandas to add its own data types for missing values. These are implemented by using two arrays instead of one. The main array represents the data in the same way as if there were no missing values. But there is an additional boolean array that indicates which values from the main array are present and need to be considered, and which are not and must be ignored.

The Apache Arrow in-memory data representation includes an equivalent representation as part of its specification. By using Arrow, pandas is able to deal with missing values without having to implement its own version for each data type.

Speed

There are many operations that can be done with dataframes. And each case would require its own analysis, but in general we can assume that the Arrow implementation is able to perform operations faster. Some examples, using a dataframe with 2.5 million rows in my laptop:

Operation	Time with NumPy	Time with Arrow	Speed up
read parquet (50Mb)	141 ms	87 ms	1.6x

Operation	Time with NumPy	Time with Arrow	Speed up
mean (int64)	2.03 ms	1.11 ms	1.8x
mean (float64)	3.56 ms	1.73 ms	2.1x
endswith (string)	471 ms	14.9 ms	31.6x

We can see how Arrow seems to be consistenly faster. And in the case of dealing with strings, the difference is huge, since NumPy is not really designed to work with strings (even if it can support them).

Interoperability

In the same way a CSV is a file format that different projects understand (pandas, R, Excel, etc), Arrow is also a program independent format. It may be less obvious, since it's an in-memory format, not a file format. But even if it doesn't have a file extension associated, you can think of it as data that can be accessed by different consumers that implement its open specification.

This has two main implications. It's (relatively) easy and standard to share the data among different programs. And it can be done in a extremely fast and memory efficient way, since two programs can literally share the same data (the same memory, without having to make a copy for each program).

This may not sound useful or intuitive, since there isn't an easy way (that I know) to use simultaneously for example pandas and R with the same loaded data. And maybe it's not common to want to do do anyway. But there are some examples where interoperability can be helpful, and we will see one of them.

Interoperability example

I need to build a pipeline to load some data from my company data warehouse, transform it, compute some analytics, and then export an automatically generated long report with the analytics. The infrastructure in my company is not very modern, and my data is available as SAS files. I want to generate my reports with a professional and scientific looking style, so I decide to use LATEX for the output.

Given the problem, pandas seems like a reasonable choice of tool for the job. It can import data from SAS files, and it can build LATEX tables. Another choice could be Polars, which is similar to pandas. Not as stable or mature yet, but it's faster and more memory efficient than pandas. This is among other things because it provides a query optimizer that can make the pipeline run faster by analyzing all operations together before executing them. Unfortunately, Polars is not able to load data from SAS or export LATEX tables as easily as pandas (at least that I know).

Besides just ignore Polars and use pandas, another option could be:

- Load the data from SAS into a pandas dataframe
- Export the dataframe to a parquet file
- Load the parquet file from Polars
- Make the transformations in Polars
- Export the Polars dataframe into a second parquet file
- Load the Parquet into pandas
- Export the data to the final LATEX file

This would somehow solve our problem, but given that we're using Polars to speed up things, writing and reading from disk is going to be slowing down my pipeline significantly.

In the old pandas world, we would appreciate if pandas could export the NumPy arrays containing the dataframe data to a memory format that Polars could understand. Or if Polars could understand NumPy directly. Both happen to be true actually. But things are even better with the new pandas. Because Polars internal data representation is actually Apache Arrow, and now it's also one of the possible internal representations for pandas dataframes.

It may seem that by using Apache Arrow I can just share a Python Arrow structure among both projects and that's it. And it really is for final users. But things are actually more complex. Both pandas and Polars are Python wrappers around other libraries, so the data to share is not really a Python structure. In the case of pandas the Arrow backed data is actually a PyArrow (C++) structure. While the Python dataframe in Polars is an wrapper around a Rust data structure, an Arrow2 Rust structure to be specific (there is another Arrow implementation in Rust, the official Arrow).

So, when sharing data between pandas and Polars, what we are really doing is to convert a PyArrow object into an Arrow2 object (or the other way round). And the good news is that there is not much to convert, since internally both libraries implement the same data representation specification, the Apache Arrow specification.

The change is not immediate, and some metadata with the pointer to where the data is stored in memory and information about the schema needs to be generated from one library and sent to the other. But the data itself (which could potentially be gigabytes or terabytes of data) stays the same, and doesn't need to be copied or converted. This allows sharing the data to happen extremely fast even when the data is huge.

So, our program could look something like this:

```
loaded_pandas_data = pandas.read_sas(fname)

polars_data = polars.from_pandas(loaded_pandas_data)
# perform operations with pandas polars

to_export_pandas_data = polars.to_pandas(use_pyarrow_extension_array=True)
to_export_pandas_data.to_latex()
1234567
```

At the time of writing this post `polars.to_pandas` has been updated just few days ago to support the `use_pyarrow_extension_array` and allow the Arrow data in Polars to be shared directly to pandas (without converting it to NumPy arrays. And `polars.from_pandas` has a bug that causes the data being copied, but it should be fixed soon. The above code will unnecessarily make a copy right now, but it should not making a copy after the release of pandas 2.0 and the next release of Polars.

Data types

One last advantage of using Apache Arrow as the container for pandas data is the support for more and better data types compared to NumPy. As a numerical computing tool, NumPy provides great support for integer and float values. It also has support for boolean values backed by 8 bits per value, and datetime values backed by 64 bits. But not much more than that.

Arrow types are broader and better when used outside of a numerical tool like NumPy. It has better support for dates and time, including types for date-only or time-only data, different precision (e.g. seconds, milliseconds, etc.), different sizes (32 bits, 63 bits, etc.). The boolean type in Arrow uses a single bit per value, consuming one eighth of memory. It also supports other types, like decimals, or binary data, as well as complex types (for example a column where each value is a list). There is a table in the pandas documentation mapping Arrow to NumPy types.

Arrow also defines a type to encode categories, so you can for example have a column with the values "red", "green" and "blue", and instead of having to store the strings for each element in the possibly huge column, an index is stored for each value (e.g. 0, 1 and 2), and then a small lookup table to know that 0 is "red", 1 is "green" and 2 is "blue". All this transparent to users, who will only notice of this data representation because of the memory saving. As a note, pandas has its own implementation of an equivalent categorical type backed by NumPy arrays.

See this example:

```
import datetime
import pyarrow
import pandas


articles = pandas.DataFrame({
    'title': pandas.Series(['pandas 2.0 and the Arrow revolution',
                            'What I did this weekend'],
                           dtype='string[pyarrow]'),
    'tags': pandas.Series([['pandas', 'arrow', 'data'],
                           ['scuba-diving', 'rock-climbing']],
                          dtype=pandas.ArrowDtype(pyarrow.list_(pyarrow.string()))),
    'date': pandas.Series([datetime.date(2023, 2, 22),
                           datetime.date(2022, 11, 3)],
                          dtype='date32[pyarrow]')
})
1234567891011121314151 6
```

As you can see by the `dtype` attributes, pandas will be storing this information in formats you may have not seen before. pandas has supported all the data in the example above since the early days, but the internal data type used was `object`. If you are not familiar with the `object` type, it is just a pointer to a Python object, and operations with the data type would probably be as slow as if you implemented your code directly using Python structures (quite slow). In comparison, the Arrow types are stored in memory with performance in mind, and operations with the data should be reasonably fast.

One question you probably have is, what operations can I do with Arrow types? And the answer depends on the exact data type. For example, Arrow strings are well supported, and have been available in pandas for few versions now. The `date32` data type is quite new, but operations like `articles['date'].dt.weekday` will work.

What about the list of strings type used for `tags`? There is nothing much supported with them right now. If you try the example above, you will see that not even printing pandas data with the data type is supported. This may sound disappointing but I have some good news. First, support for some Arrow types has just been introduced, and it'll be improved over time. And second, nothing prevents you from writing your own operations with any language that has an Apache Arrow implementation. And as we'll see in the part II of this article, it is not difficult.

Summary

In this article we've seen how Apache Arrow and its integration into pandas is helping pandas be simpler, run faster, play better with related libraries, and efficiently represent more types of data.

In part II we will show how to implement pandas extensions by using Arrow. These extensions can run extremely fast, and not only work for pandas, but also to work for any other library using Apache Arrow internally (such as Polars or Vaex).